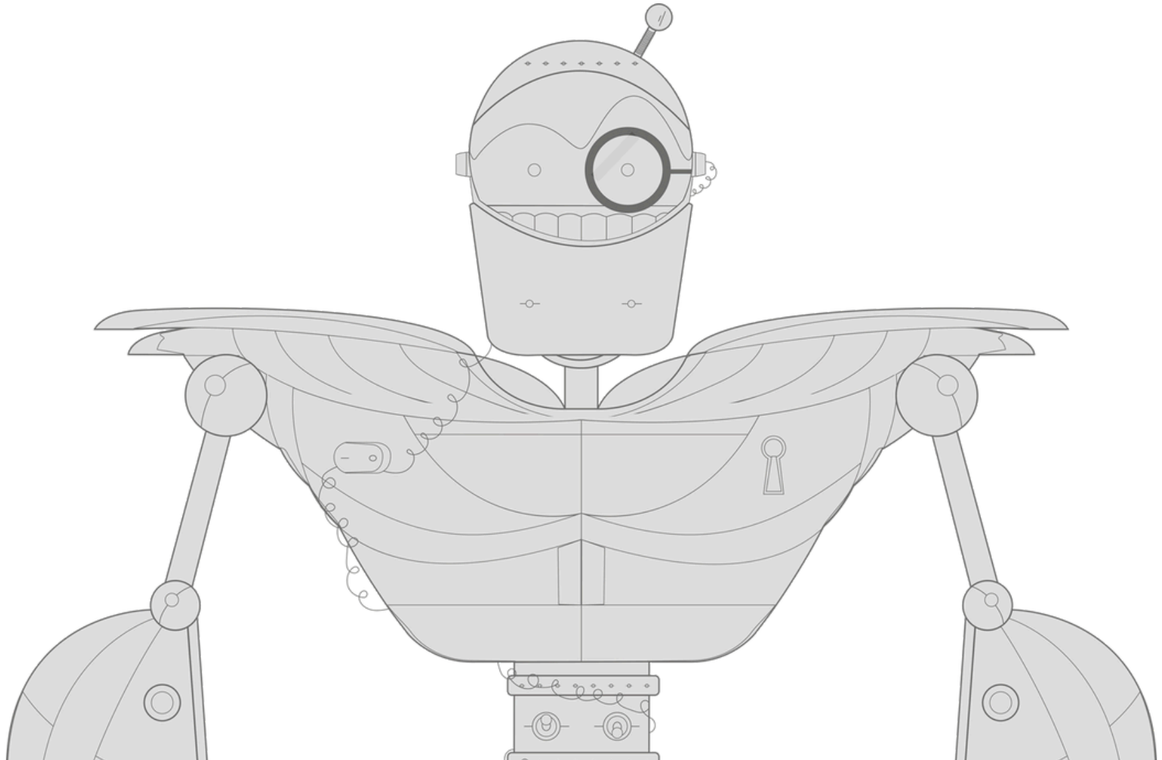
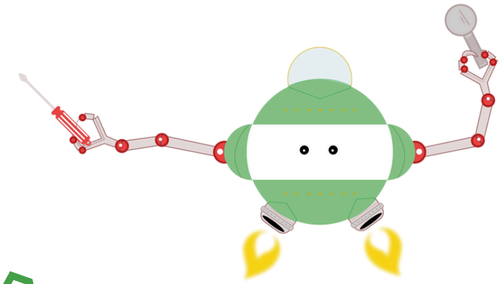
Labforweb
nerd academy

Funzioni Parametriche

A cura di Gaia Moschetto





IN 1/2 'ORA

Funzioni Parametriche

Come già visto, una funzione è un insieme di istruzioni che può essere utilizzato più volte all'interno del codice, per evitare ripetizioni.

Inoltre, è possibile passare delle informazioni a una funzione, ovvero dei ***valori di ingresso*** sui quali verranno eseguite determinate operazioni.

Funzioni Parametriche

In questo senso quindi, i **parametri** rappresentano dei ***valori variabili*** che permettono di "*personalizzare*" una funzione, che di conseguenza potrà essere utilizzata in molti contesti.
Così facendo, sarà sufficiente scrivere il codice **una sola volta** e poi, per poterlo eseguire su elementi diversi, individuare le porzioni variabili.

Funzioni Parametriche

I parametri vengono specificati nel momento della **definizione** della funzione, e svolgono a tutti gli effetti il ruolo di *variabili*.

```
function moltiplicazione(fattore1, fattore2){
   console.log(fattore1 * fattore2);
}
```

Attenzione, perchè in questa fase tali parametri **non** hanno un valore definito, rappresentano semplicemente dei *segnaposto*: assumono valore nel momento in cui la funzione viene richiamata.

```
function moltiplicazione(fattore1, fattore2){
   console.log(fattore1 * fattore2);
}

moltiplicazione(5, 9);
```

Funzioni Parametriche

```
function moltiplicazione(fattore1, fattore2){
   console.log(fattore1 * fattore2);
}

moltiplicazione(5, 9);
```

Quando la funzione viene richiamata, i valori passati vengono copiati nelle variabili locali **fattore1** e **fattore2**, che verranno utilizzate nel *console.log()*.

Funzioni Parametriche

```
function moltiplicazione(fattore1, fattore2){
   console.log(fattore1 * fattore2);
}


moltiplicazione(5, 9);

moltiplicazione(1, 7);

moltiplicazione(45, 2);
```

La funzione *moltiplicazione()* viene richiamata 3 volte, ed in ciascun richiamo i parametri *fattore1* e *fattore2* assumono un valore diverso:

- 1° richiamo - **fattore1** = **5** & **fattore2** = **9**
- 2° richiamo - **fattore1** = **1** & **fattore2** = **7**
- 3° richiamo - **fattore1** = **45** & **fattore2** = **2**

Funzioni Parametriche

```
function moltiplicazione(fattore1, fattore2){
   console.log(fattore1 * fattore2);
}


moltiplicazione(5, 9);

moltiplicazione(1, 7);

moltiplicazione(45, 2);
```

In console vedremo quindi 3 valori numerici differenti, uno per ciascun richiamo, in ordine:
45, **7** e **90**

Funzioni Parametriche

```
function stampaNome(nome){
   console.log('Il mio nome è ' + nome);
}

stampaNome('John');
```

La parola "**nome**" è il parametro, ovvero un *segnaposto* in attesa di ricevere un valore, che verrà passato durante la chiamata della funzione.
La stringa '*John*' è il valore che il parametro "*nome*" assume.
Prima del richiamo di *stampaNome()*, "*nome*" non ha alcun valore, è uno spazio di memoria vuoto.

Funzioni Parametriche

Se non viene fornito alcun parametro, questo assume valore **undefined**.

La seguente funzione ad esempio, può essere richiamata con un solo parametro, nonostante nella sua definizione ce ne siano 2:

```
function mostraMessaggio(mittente, messaggio) {

  mittente = '*' + mittente + '*';

  alert(mittente + ': ' + messaggio);
}

mostraMessaggio('Mario Rossi');
```

Funzioni Parametriche

Ciò non rappresenta un errore, semplicemente una chiamata simile mostrerà "***Mario Rossi*: undefined***". Infatti, dal momento che non viene passato alcun valore a *"messaggio"*, questo risulterà **undefined**.

```
function mostraMessaggio(mittente, messaggio) {

  mittente = '*' + mittente + '*';

  alert(mittente + ': ' + messaggio);
}

mostraMessaggio('Mario Rossi');   //Output: '*Mario Rossi*: undefined'
```

Funzioni Parametriche

In questi casi, è possibile specificare un cosiddetto **valore default** per i parametri nella dichiarazione di funzione, usando il simbolo **=**, in questo modo:

```
function mostraMessaggio(mittente, messaggio = 'testo non presente') {

  mittente = '*' + mittente + '*';

  alert(mittente + ': ' + messaggio);
}

mostraMessaggio('Mario Rossi');   //Output: '*Mario Rossi*: testo non presente'
```

Adesso, se il parametro ***messaggio*** non viene passato, assumerà il valore "***testo non presente***", ovvero una *stringa*, ma in realtà potrebbe assumere un qualsiasi valore (*numero*, *booleano*, *funzione*, *espressione complessa*, ...).

Funzioni Parametriche

E se richiamiamo la funzione con più parametri rispetto quelli richiesti?

JavaScript semplicemente ignora gli ulteriori parametri inseriti.

```
function miaFuzioneConParametri(a,b) {

  console.log("Mia funzione - parametro a = " + a);

  console.log("Mia funzione - parametro b = " + b);
}

miaFuzioneConParametri(1, "due", 3, 4, true);
```

Rest Parameters ...

Una funzione può essere richiamata con un numero qualsiasi di argomenti, indipendentemente da come è stata definita.

```
function sum(a,b) {

  return a + b;
}

console.log(sum(2, 5, 8, 10, 1));
```

In questo esempio, non ci saranno errori dovuti ad argomenti "eccessivi", ma ovviamente nel risultato della somma saranno conteggiati solo i primi 2, che quindi sarà 7.

Rest Parameters ...

Il resto dei parametri può essere incluso nella definizione della funzione utilizzando 3 punti (. . .) seguiti dal nome dell'**array** che li conterrà.
Questa sintassi significa letteralmente "*raccogli i parametri rimanenti in un array*".

```
function sum(...numeri) {

  let sum = 0;

  for (let i=0; i < numeri.length; i++) {
    sum += numeri[i];
  }

  return sum;
}

console.log(sum(2, 5, 8, 10, 1));  // 26
console.log(sum(1, 6));  // 7
console.log(sum(5));  // 5
```

Rest Parameters ...

I rest parameters raccolgono tutti gli argomenti rimanenti, quindi l'istruzione **...rest** deve essere sempre l'ultima.
Infatti, quanto segue non ha senso e genera un errore:

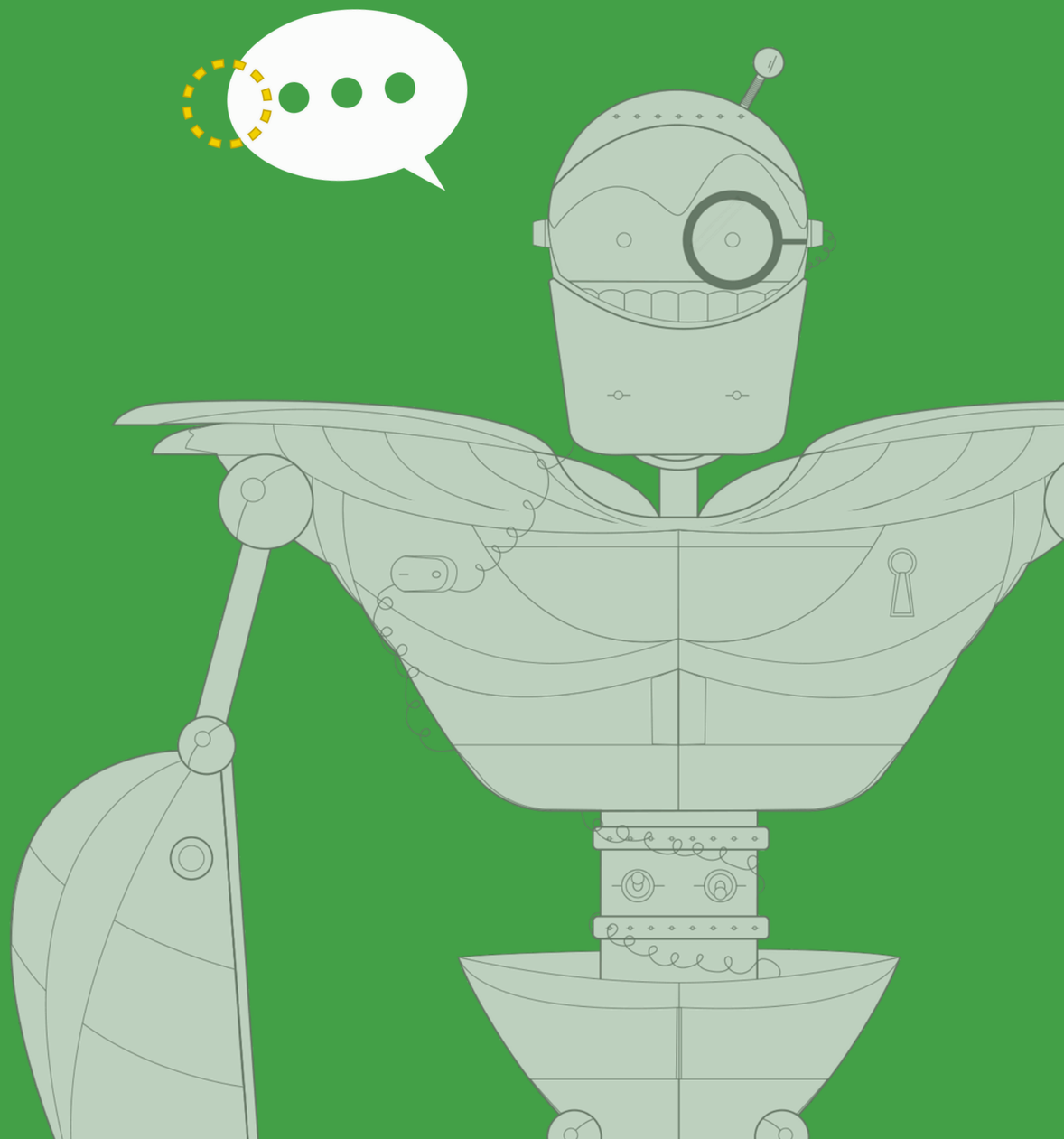
```
function test(arg1, ...numeri, arg2) {

  //ERRORE

}
```

Ricapitolando

- I parametri sono delle variabili utilizzabili all'interno della funzione, che vengono effettivamente inizializzate all'atto della chiamata della funzione stessa.
- È possibile inizializzare i parametri di una funzione con dei **valori di default**: questi valori verranno utilizzati se non è stato specificato nessun valore all'atto della chiamata.
- Se richiamiamo la funzione con più parametri rispetto quelli richiesti, quelli "di troppo" verranno semplicemente ignorati.
- La sintassi **rest parameters** consente di rappresentare un numero indefinito di argomenti come un array, in questo modo una funzione può essere chiamata con qualsiasi numero di argomenti, indipendentemente da come è stata definita.

Labforweb
nerd academy

Domande...



Grazie

www.labforweb.it